

Correcciones y mejoras

Compras · Frontend (Angular) + Backend (Spring Boot)

Del sintoma a la causa: que se cambio, por que, y que regla deja cada caso.

Este documento junta cuatro sesiones de trabajo del mismo día. Cada una arranco con un sintoma visible en pantalla y termino en una causa que no era la que se sospechaba. El valor de leerlas no esta en el COMO se resolvió, sino en el POR QUE: las cuatro son el mismo tipo de error, repetido en cuatro ropa distintas.

1. Las cuatro sesiones de un vistazo

SESION	SINTOMA	CAUSA REAL
V8 Alineacion	Titulos de columna corridos	Pelea de especificidad CSS
V9 Validaciones	Guardar no hacia nada, sin aviso	El validador creía el vacio valido
V10 Modal cajas	El formulario pisaba la tabla	Flex encogia por debajo de su contenido
BK Flyway	El backend no arrancaba	El checksum incluye los comentarios

Hay un hilo comun, y es el que las cuatro veces se paso por alto: en los cuatro casos el problema NO estaba donde se miraba. Se miraba el CSS y era elHTML; se miraba el HTML y era el validador; se miraba el CSS del modal y era el `min-height: auto` de los inputs; se miraba la base de datos y era el historial de Flyway.

LA LECCION QUE SI SE TRANSFIERE

Cuando el sintoma aparece en un archivo y la causa esta en otro, la pregunta que hay que hacerse no es "como arreglo esto" sino "por que funcionaba antes y ahora no, o nunca funciono". La segunda pregunta siempre lleva al archivo correcto.

2. V8 — Alineacion: la pelea de especificidades CSS

2.1 El sintoma era siempre el mismo, en cuatro tablas distintas

Los titulos de las columnas no caian encima de sus datos. En unos casos el titulo se iba a la derecha y el numero al centro; en otros el titulo se quedaba a la izquierda mientras los botones de acciones quedaban al centro. Cuatro tablas, cuatro sintomas parecidos, y la primera reaccion razonable es "cada tabla tiene su propio problema".

La primera sorpresa: las cuatro tenian el mismo problema. Y no era de las tablas.

2.2 El global ya centraba; el problema era quien le ganaba

css — el mismo nombre de clase dando dos alineaciones distintas

```
/* styles.css – ya existia antes de tocar nada */
.table-card th,
.table-card td {
  text-align: center;
}

/* ...pero cada componente redefinía el alineamiento de sus montos: */
.money          { text-align: right; } /* peso (0,1,0) -> PIERDE al global */
th.money        { text-align: right; } /* peso (0,1,1) -> empate, gana por ORDEN */

.sesiones-table .money { text-align: right; } /* peso (0,2,0) -> GANA */
.sesiones-table thead th { text-align: left; } /* peso (0,1,2) -> GANA */
```

EL CASO QUE LO EXPLICA TODO

En el historial de ventas el `<td>` usaba `.money` y su `<th>` también. El `<td>` lo resolvía `.table-card td` (peso 0,1,1), que le gana a `.money` (0,1,0) y lo dejaba centrado. El `<th>` lo resolvía `th.money` (0,1,1): empate EXACTO con `.table-card th`, y en un empate gana el que aparece después en el CSS. Título a la derecha, número al centro. Ese era el título corrido.

2.3 La solución y su trampa

La corrección fue declarar ``center`` de forma explícita en el ``th`` y en el ``td`` de las cuatro tablas, más padding simétrico a los dos lados.

- El ``center`` va en los dos porque apoyarse en el global sería depender del orden de ``styleUrls``, que es frágil: si mañana se mueve un archivo, la tabla se descuadra sola.
- El padding paso a simétrico: con el texto centrado, un padding de un solo lado lo descentra hacia el otro.
- Los ``!important`` se conservaron: ya peleaban con el padding del global, y quitarlos sería un segundo cambio en la misma línea.

Y lo que NO se tocó también importa: las subtablas de detalle, el encabezado del carrito del modal y el total grande del modal. No son columnas de las cuatro tablas pedidas. Un total de modal a la derecha está bien; centrarlo sería peor.

REGLA

Cuando dos reglas empatan en especificidad, gana el orden del archivo. Si dos archivos CSS tocan el mismo elemento, declaralo en los dos con el mismo peso y el mismo valor: la duplicación es más barata que un bug que reaparece cuando alguien reordena un import.

3. V9 — Validaciones: el return silencioso

3.1 Cinco formularios, un solo defecto

Cinco formularios frenaban el guardado con un ``return`` y ninguna pista. El usuario apretaba Guardar, no pasaba nada, y no tenía forma de saber si le habían guardado, si el botón estaba roto o si el guardado había fallado.

Las cinco paradas sin aviso

```
// categorias.ts, roles.ts, compras.ts – el patron repetido
if (!this.categoryName.trim()) return;

// usuarios.ts – la misma idea, con snackbar
if (password.trim().length < this.MIN_PASSWORD_LENGTH) {
  this.snack.open(`La contraseña debe tener mínimo 8 caracteres`, ...);
  return;
}
```

3.2 La causa comun: los `<span>` existian, pero nadie escribia en el mapa

Aqui esta lo importante, y es lo que hizo que el defecto pasara inadvertido tanto tiempo: los cinco formularios YA TENIAN el markup completo. Habia un `<span class="error-msg">`, y en productos el input con `[class.input-error]`. El HTML estaba bien.

validadores.ts — el vacio se trata como valido

```
// core/utis/validadores.ts (ANTES) – no hay ningun flag de obligatoriedad
validarTexto(valor, campo, max = 100) {
  const v = (valor || '').trim();

  if (!v) return ok(); // el vacio PASA como valido
  if (v.length > max) return fail(`El ${campo} no puede tener más de ${max} caracteres.`);
  return ok();
}
```

Y el encadenamiento era este:

La cadena rota: validar OK, borrar el error, frenar sin avisar

```
validarNombre() {
  const r = validarTexto(this.categoryName, 'nombre de la categoría', 50);
  if (r.ok) delete this.errores['name']; // borra el error...
  else      this.errores['name'] = r.error;
}

saveCategory() {
  if (!this.categoryName.trim()) return; // ...y despues frena en silencio
}
```

POR ESO EL MENSAJE "NO APARECIA"

El HTML estaba bien, el validador tambien, por separado. El que fallaba era el encadenamiento: validar devolvía ok con el vacío, el `delete` se llevaba el error, y el `*ngIf` del span se quedaba con la condición falsa. No era un problema de CSS ni de Angular ni del navegador.

3.3 Los flags que faltaban

La correccion fueron parametros opcionales en los validadores compartidos. Todos con un default que conserva el comportamiento anterior, para no romper a los doce servicios que ya los usan.

Las seis firmas despues del cambio

```
validarTexto(valor, campo, max = 100, obligatorio = false)
validarEmail(valor, obligatorio = false)
validarPassword(valor, obligatorio = false)
validarRfc(valor, obligatorio = false)
validarEntero(valor, campo, min = 0)
validarPrecio(valor, campo, min = 0)
```

El flag `obligatorio` se resuelve con una funcion auxiliar que decide el genero del mensaje, porque "La contrasena es obligatorio" esta mal y un detalle asi se nota:

validadores.ts — el genero del mensaje

```
function mensajeObligatorio(campo, femenino = false) {
  return femenino
    ? `La ${campo} es obligatoria.`
    : `El ${campo} es obligatorio.`;
}
```

4. V9 — Tres detalles que hacen que la validacion funcione de v

4.1 `min = 0` por default ES la decision correcta

Al agregar `min` a `validarEntero` y `validarPrecio` lo facil habria sido poner `min = 1` por default. Eso habria roto el formulario de caja.

El otro consumidor de validarPrecio

```
// features/charge/facade/cash-facade.ts
const resultado = validarPrecio(this.openingAmount, 'monto inicial');
if (!resultado.ok) { ... }
```

EL MONTO INICIAL DE CAJA PUEDE SER CERO

Una caja puede abrirse sin fondo: el cajero abre el turno y deposita despues. Con min = 1 como default, esa operacion legitima quedaria bloqueada por un cambio pensado para productos. El default se queda en 0 y es productos quien pide min = 1 de forma explicita.

Y el corolario, que es la parte fina: el stock `0` SI es valido en la base. Un producto que se vendo hasta agotarse queda en 0, y ahi no es un error: es el estado normal de un producto agotado. Lo que no se permite es CREARLO en 0. Por eso el `min = 1` va en el formulario, no en el servicio ni en el backend.

4.2 `novalidate` es lo que hace que el aviso aparezca

El formulario de productos paso de `min="0"` a `min="1"`. Pero solo con `min` el resultado seria el equivocado: el navegador bloquea el submit por su cuenta.

productos.html — el nombre, que antes ni se validaba

```
<form class="form-card" (ngSubmit)="save()" novalidate>
  <input
    name="name"
    [(ngModel)]="form.name"
    (input)="validarCampo('name')"
    [class.input-error]="errores['name']"
  />
  <span class="error-msg" *ngIf="errores['name']">{{ errores['name'] }}</span>
</form>
```

DOS AVISOS PELEANDO POR EL MISMO SUBMIT

Sin novalidate, el min="1" del navegador detiene el formulario antes de que Angular vea nada. El usuario ve un globito gris del navegador, en ingles, que no dice que campo es el culpable. Gana el nativo porque ocurre primero, y nuestro mensaje en rojo nunca se pinta. `novalidate` no desactiva la regla: el `min` sigue sirviendo para el spinner y el teclado, solo le quita la autoridad de frenar el formulario.

4.3 El orden: mayusculas antes de validar, en un solo metodo

Productos y compras tenian DOS atributos `(input)` en la misma etiqueta, uno para pasar a mayusculas y otro para validar. El RFC incluso tenia tres. Angular los dispara en orden de plantilla, pero `ngModel` actualiza su propio valor por su cuenta y sin avisar.

De dos (input) sueltos a un metodo con el orden escrito

```
<!-- compras.html (ANTES) -->
<input
  name="formProvider.rfc"
  [(ngModel)]="formProvider.rfc"
  (input)="aMayusculas($event)"
  (paste)="onPasteRfc($event)"
  (input)="validarCampo('rfc')" // depende del orden
/>

// compras.ts (AHORA) — un metodo, orden explicito
onInputRfc(event: Event) {
  aMayusculas(event); // 1. mayusculas primero
  this.validarCampo('rfc'); // 2. validar despues
}
```

POR QUE IMPORTA EL ORDEN

Si la validacion corre antes de las mayusculas, el error se calcula contra el texto en minusculas de `formProvider.rfc` mientras la caja ya muestra mayusculas. El mensaje describe un valor que el usuario nunca vio. Mismo patron en `onInputSku()` y `onInputProviderName()`.

4.4 El `0` del stock: el que escribe, decide

La correccion silenciosa

```
// productos.ts – metodo que el template YA NO llama
validateNumber(field) {
  const min = field === 'stock' ? 0 : 1;
  if (this.form[field] < min) {
    this.form[field] = min;      // corrige en silencio: el 0 se vuelve 1
  }
}
```

Escribir 0 en el stock y ver la caja saltando a 1 sin aviso es PEOR que un error de validacion: el usuario cree que escribio lo que quiso, el campo muestra 1, y no tiene idea de por que. Se desacoplo la correccion del aviso: el template ya llama a `validarCampo('stock')`, que avisa y deja el 0 que el usuario escribio.

4.5 Los estilos: dos reglas duplicadas, subidas al global

REGLA	DONDE ESTABA	AHORA
.error-msg	productos.css y compras.css (x2)	styles.css
.input-error	productos.css y compras.css (x2)	styles.css

Los cinco formularios necesitan las dos reglas y ninguno de los dos CSS de componente cubria a los otros tres. Se movieron a `styles.css`, donde viven las demas reglas compartidas. El `!important` en el borde se conserva a proposito: los formularios declaran su propio `border-color` con la MISMA especificidad que `.input-error`, asi que sin el ganaria el del componente y el input se veria normal, con el mensaje debajo pero sin el campo marcado.

FORMULARIO	ANTES	AHORA
categorias.ts	return silencioso	validarNombre() y corte si hay error
roles.ts	return silencioso x2	validarNombre() en crear y actualizar
usuarios.ts	snackbar de contrasena	validarFormulario() en el campo
compras.ts	return si falta nombre o RFC	validarFormularioProveedor()
productos.ts	nombre sin validar	nombre + min = 1 en precio y stock

UN DETALLE QUE SE VIO DE PASO

El mensaje de roles.ts decia "El nombre de la categoria no puede tener mas de 50 caracteres": era un copy-paste de categorias.ts, y el usuario estaba leyendo sobre su ROL. Ademas el maxlength del input era 30 y el validador pedia 50, o sea que la regla del largo no se podia disparar nunca. Las dos cosas se corrigieron juntas.

5. V10 — Modal de cajas: por que el formulario pisaba la tabla

5.1 No eran dos modales

La primera interpretacion fue "estos dos modales se enciman". No era eso: es UN modal, con tres hijos, y el problema era como se repartian el alto.

css — donde estaba el defecto

```
.modal {
  display: flex;
  flex-direction: column;
  max-height: calc(100vh - 48px);
}

.modal h3      { flex: 0 0 auto; } /* el h3 ya estaba protegido */
.box-form     { /* SIN flex */ }  /* por defecto: 0 1 auto = ENCOGE */
.modal-body   { flex: 1 1 auto; overflow-y: auto; }
```

Por defecto un hijo flex es `flex: 0 1 auto`: **encoge**. Con muchas cajas el alto total pasa el `max-height` y los dos bloques empiezan a encogerse. `.modal-body` absorbe el escurrimiento porque tiene scroll, pero `.box-form` tambien encoge.

AQUI ESTA EL BUG DE VERDAD

Los `<input>` del formulario son hijos flex con `min-height: auto`, o sea NO pueden bajar de su alto natural. Si el formulario encoge por debajo de ese alto, su contenido se sale de su caja y PISA la tabla de abajo. El `h3` del modal ya tenia `flex: 0 0 auto` por exactamente esta razon; al `.box-form` le faltaba.

cajas.css — la correccion completa

```
.box-form {
  ...
  flex: 0 0 auto; /* una linea: el que cede espacio es .modal-body */
}
```

5.2 Bloquear la tabla mientras se crea o edita una caja

El usuario pidio que el modal de la tabla no se pudiera usar mientras se crea una caja. Parece un detalle visual, pero es un bug de datos.

El bloqueo, en las tres capas

```

<!-- cobro.html -->
<div class="modal-body" [class.tabla-bloqueada]="cash.showBoxForm">

  <button class="box-btn-edit"
    [disabled]="cash.showBoxForm"
    (click)="cash.startEditBox(b)">Editar</button>

  <!-- y los otros tres: del / baja / alta -->
</div>

<!-- Pie del modal: los DOS se inhabilitan -->
<button class="btn-confirm-open" [disabled]="cash.showBoxForm"
  (click)="cash.startNewBox()">Crear</button>
<button class="btn-cancel" [disabled]="cash.showBoxForm"
  (click)="cash.closeBoxesModal()">Cerrar</button>

```

Si el usuario estaba escribiendo "CAJA 3" y aprieta "Editar" sobre otra fila, `startEditBox` reemplaza `cash.boxForm` por debajo y se pierde lo tecleado sin aviso. Con "Crear" pasa lo mismo, y con "Cerrar" se pierde el formulario entero.

CAPA	QUE RESUELVE
.tabla-bloqueada	pointer-events: none: bloquea cualquier clic del bloque, incluidas filas futuras
[disabled]	Saca el control del recorrido con teclado y lo pone en gris de forma nativa
opacity: 0.35	Marca visualmente el bloque como inactivo sin esconderlo

La tabla NO se oculta a proposito: el usuario necesita ver sobre que caja esta editando y en que estado estan las demas. Lo que se apaga es su capacidad de actuar. Y quedan dos salidas explicitas, "Cancelar" y "Guardar": el usuario no queda atrapado.

5.3 Centrar el texto de un boton que es flex

La segunda peticion fue centrar el texto de los botones de `.button-box`. La causa fue el tipo de display.

cobro.css — la diferencia entre text-align y justify-content

```

.btn-confirm-open {
  display: inline-flex; /* el texto NO es un bloque: es un flex item */
  align-items: center;
  justify-content: center; /* esto es lo que centra el texto */
  text-align: center; /* y esto aqui NO hace nada */
}

```

POR QUE TEXT-ALIGN NO CENTRABA NADA

`text-align` solo actua sobre CAJAS DE LINEA. En un contenedor flex el texto es un flex item, y los flex items se colocan con `justify-content`. Con solo `text-align` el texto se quedaba pegado al padding izquierdo, con un hueco a la derecha: exactamente lo que se veia. El mismo razonamiento ya estaba aplicado en los `.box-btn-\*` de cajas.css.

Un detalle de alcance: `.btn-confirm-open` es la MISMA clase en los tres botones del archivo (Crear caja, + Crear caja, Abrir). Arreglarla una vez los centro a los tres. No hacia falta una regla duplicada en `cajas.css` para la seccion de cajas.

Y de paso, dos cosas que aparecieron al mismo tiempo: el `:hover` sin `:not(:disabled)` hacia que un boton deshabilitado siguiera cambiando de color al pasarle el mouse, y un `;;` doble en `background: #22c55e`.

6. Backend — Flyway y los comentarios

6.1 El sintoma

El backend dejo de arrancar con `Validate failed: Migrations have failed validation`, con un mismatch de checksum en las migraciones 4, 5, 6 y 7. El mensaje decia "revert the changes to the migration, or run repair", y el usuario no habia tocado ninguna sentencia: solo habia quitado comentarios.

6.2 La causa: el checksum incluye los comentarios

El checksum de Flyway se calcula sobre el TEXTO COMPLETO del archivo, linea por linea, comentarios incluidos. Editar un comentario cambia el checksum aunque el SQL haga exactamente lo mismo.

ARCHIVO	UNICO CAMBIO
V4__cajas_reutilizables.sql	quito & a 2 lineas de comentario
V5__ventas_usuario...sql	quito & a 1 linea de comentario
V6__ventas_idempotencia.sql	quito & a 1 linea de comentario
V7__campos_obligatorios.sql	quito Ø=Ý a 1 linea y & a 2 lineas

Cero sentencias ejecutables cambiadas. Flyway, sin embargo, no puede saber que la diferencia es cosmetica, asi que aborta arranque por si acaso. Es el comportamiento correcto: un cambio real en una migracion aplicada es un problema mucho mas grave que un arranque fallido.

LO IMPORTANTE: EL ERROR NO ERA DEL ESQUEMA

La primera pregunta correcta no era "como reparo el arranque" sino "esta mal la base o esta mal el historial". Antes de escribir nada se consulto `flyway_schema_history` en solo lectura: 7 migraciones, todas `success = t`, ninguna fallida, ninguna pendiente. El esquema estaba perfecto; lo unico viejo eran 4 numeros en una tabla de bookkeeping.

6.3 La reparacion

Salida real del repair

```
$env:MAVEN_ARGS = 'flyway:repair ...'
mvn flyway:repair

[INFO] Repair of failed migration ... not necessary.
       No failed migration detected.
[INFO] Repairing Schema History table for version 4 ... 1384467832
[INFO] Repairing Schema History table for version 5 ... -184526483
[INFO] Repairing Schema History table for version 6 ... -1358097441
[INFO] Repairing Schema History table for version 7 ... -1130579926
[INFO] Successfully repaired schema history table.
```

El "No failed migration detected" es la garantía de que no se borro historial ni se marco nada como aplicado: `repair` tambien hace esas dos cosas, y aqui no hizo ninguna.

Se agrego `flyway-maven-plugin` al `pom.xml` porque sin el no habia forma de hablar con la BD sin arrancar la app, y la app es justamente lo que no arranca. El plugin se declaro SIN credenciales, porque ese archivo si se sube a git.

UN DETALLE DEL POM QUE SI IMPORTABA

El <dependency> de un <plugin> NO hereda del <dependencyManagement> del POM (el primer intento falló con "version is missing"). Se resuelve con `\${flyway.version}`, la propiedad que define el BOM de Spring Boot, para que el modulo siga pegado a la misma version de Flyway que usa la app. Si se pusiera un numero fijo, Maven y la app podrian calcular checksums con algoritmos distintos, y el problema volveria.

6.4 La trampa de Windows

Por que los ^ no son opcionales en este proyecto

```
# mvn es un .cmd: cmd RE-PARSEA la linea. Sin escapar, falla:
mvn flyway:info -Dflyway.url=...?sslmode=require&prepareThreshold=0 ...
#   -> "prepareThreshold no se reconoce como un comando"

# El password contiene un >, que cmd lee como REDIRECCION.
# Con MAVEN_ARGS y escapes ^ el comando funciona:
$env:MAVEN_ARGS = 'flyway:info -Dflyway.url=...?sslmode=require^&prepareThreshold=0 ...'
mvn flyway:info
```

Si esto hay que repetirlo mucho, lo correcto es mover `flyway.url`, `flyway.user` y `flyway.password` a un perfil de `~/.m2/settings.xml`, que ni PowerShell ni cmd interpretan.

LA REGLA DEL BACKEND

Nunca editar una migracion YA APLICADA. Si el cambio es solo de comentarios, `repair` es seguro. Si cambia SQL de verdad, hay que escribir una migracion nueva (V8__...): usar repair ahi re-baselinea el checksum de algo que la base nunca ejecuto, y el defecto se descubre mucho despues y con otros sintomas.

7. Verificacion

QUE SE REVISO	RESULTADO
ng build --configuration development	OK
categorias / roles / usuarios / compras	Avisan en el campo, sin return mudo
productos	Nombre, precio y stock; min = 1
Modal de cajas	Formulario sin encimarse; tabla y pie bloqueados
Centrado de .btn-confirm-open	OK en los tres botones que lo usan
flyway:repair	4 checksums re-baselineados
flyway:info	7 migraciones, todas Success, 0 Pending
mvn spring-boot:run	Started ComprasApplication in 21.099 seconds
mvn test	No encuentra specs (ver abajo)

Los tests del frontend no corren por el entorno, no por el código: la ruta del repositorio tiene parentesis ("Sistema de ventas (comida)") y eso rompe el patron que busca los archivos de prueba. Es un problema conocido de esta maquina.

8. Las seis reglas que quedaron

Las cuatro sesiones produjeron seis reglas. Ninguna es propia de Angular, CSS ni Flyway: son todas sobre el orden en que se investiga un defecto.

REGLA	DE DONDE SALE
Un return silencioso es indistinguible de un no-op	V9 validaciones
Si dos archivos CSS tocan lo mismo, duplica la regla: el orden es fragil	V8 alineacion
Un hijo flex con contenido de altura fija necesita flex: 0 0 auto	V10 modal
text-align no centra flex items: es justify-content	V10 botones
El orden de dos eventos del mismo tag no es una garantia	V9 mayusculas y validacion
Si el historial discrepa del archivo, repara el historial, no el esquema	BK flyway

EL PATRON COMUN, SI HUBIERA QUE DEJAR UNA SOLA IDEA

En los cuatro casos el sintoma señalaba un archivo y la causa estaba en otro. La herramienta que funciono no fue intuition sino una pregunta repetida: "este comportamiento, cuando funcionaba bien, que era distinto?" El diff, el historial de Flyway y el peso de una regla CSS respondieron las tres primeras. La cuarta, la del `min-height: auto`, salio de mirar los defaults de flex en vez de los valores declarados.

Y hay un quinto caso, el mas facil de dejar pasar, que conviene repetir aqui: en usuarios la contrasena SI se validaba, y el aviso SI aparecia, pero en un snackbar que tapaba el formulario, se iba solo a los 4 segundos y no senalaba el campo culpable. Para un formulario de un campo quizas bastaba. Para uno de tres, el aviso tiene que estar pegado al campo.